

Terraform challenge

Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- The service will be a simple web server that returns a static page.
- You can use **apache2** or **nginx** to serve a simple HTML file.
- The service should be **containerized** and stored in a **Docker registry**.
- The service should be deployed on a **single VM instance** as a docker container.
- The image should be pulled from the registry specified earlier.
- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

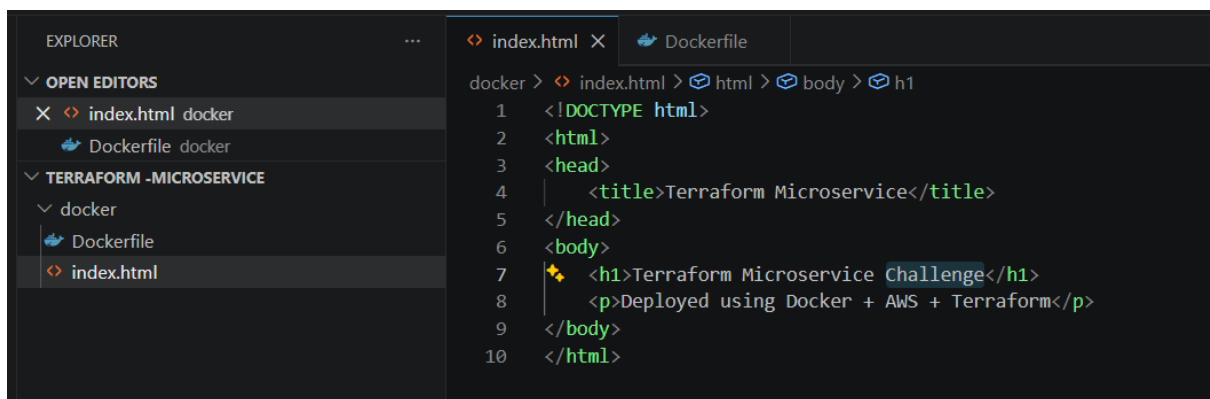
You are to provide the following as part of your solution:

1. **Docker file** to build the image.
2. **Terraform code base** to deploy the following:
 - Docker registry
 - VPC and all required networking resources
 - EC2 instance
 - Load Balancer

Notes

- Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.
- To push the image to the Docker registry, **build the image locally** before running the Terraform code.
- Once the image registry is created, use Terraform to **push the local image** to the registry.

- Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.
- For SSH access:
 - Use a locally created key.
 - Add it to the cloud provider key storage.
 - Attach it to the instance.
- Use **Terraform best practices** for file structure and code organization.
 - Use variables, locals, modules, etc. when appropriate.
- Include all tfvars, scripts, and other helper files used to deploy the application.
- You may use any cloud provider of your choosing, although **AWS is preferred**.
- Use a **free tier account** or **atrial account** to deploy the resources
- **Created Docker Folder in that index and docker file created**



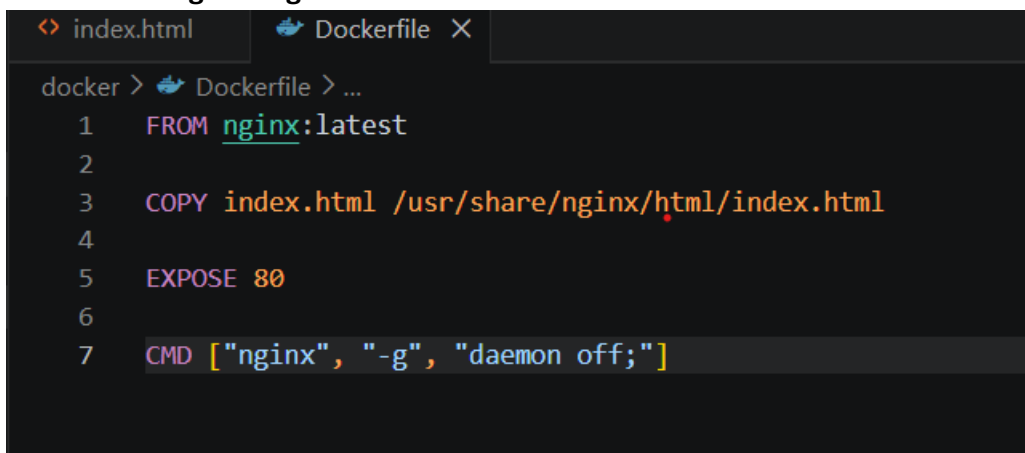
The screenshot shows a VS Code editor with two files open: `index.html` and `Dockerfile`. The `index.html` file contains the following HTML code:

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Terraform Microservice</title>
5 </head>
6 <body>
7   <h1>Terraform Microservice Challenge</h1>
8   <p>Deployed using Docker + AWS + Terraform</p>
9 </body>
10 </html>

```

- **In Docker file given Nginx**



The screenshot shows a VS Code editor with the `Dockerfile` file open. The Dockerfile contains the following instructions:

```

1 FROM nginx:latest
2
3 COPY index.html /usr/share/nginx/html/index.html
4
5 EXPOSE 80
6
7 CMD ["nginx", "-g", "daemon off;"]

```

- **Build terraform using docker**

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> cd docker
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker> docker build -t terraform-web:v1 .
ERROR: failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the service is running.
e/dockerDesktopLinuxEngine; The system cannot find the file specified.
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker> docker build -t terraform-web:v1 .
[+] Building 296.7s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 157B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 247B
=> [1/2] FROM docker.io/library/nginx:latest@sha256:6415da96b72a2f6ff433053df1da4bd507e45bc44b2c83dbac0ddeea80c6f066
=> => resolve docker.io/library/nginx:latest@sha256:6415da96b72a2f6ff433053df1da4bd507e45bc44b2c83dbac0ddeea80c6f066
=> => sha256:868d78dceaedbae0f3297cfec4de550203fe2e711d1fd6bf08d55e5aec5c0960 1.21kB / 1.21kB
=> => sha256:c82e83b01f69cc20dce86e9d334fd0ecf09ce30233bdbc1698d82bde65e986fc 1.40kB / 1.40kB
=> => sha256:99181f19640fe66e1bdb5fcc2d2b9ae60910350eba8a55ccd1c00d4c3fe01ffc 404B / 404B
```

- **Now check image and now I run docker (or) push to my docker hub**

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qfs7pltd10z2xtwesoiwfz94
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
terraform-web:v1	68d70332d2dc	238MB	63.1MB	
tomcat:latest	f8d2287df1bb	589MB	158MB	

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker> docker run -d --name web-test -p 8080:80 terraform-web:v1
a3add3f8601de4af5fde2bc0658857c126fdf49b6d0bac8e724612bb31e1b49a
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a3add3f8601d	terraform-web:v1	"/docker-entrypoint..."	15 seconds ago	Up 13 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web-test

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice\docker>
```

- **Now check in my docker hub it pushed or not**

The screenshot shows the Docker Desktop interface with the 'Images' tab selected. It displays two local images: 'tomcat' and 'terraform-web'. The 'tomcat' image is marked with a green dot, signifying it has been pushed to Docker Hub. The 'terraform-web' image is marked with a blue dot, indicating it is only present locally. The interface also shows disk usage and a search bar.

- **Now check in Browser with port 8080**



Terraform Microservice Challenge

Deployed using Docker + AWS + Terraform

- Now Create Terraform Provider Configuration

```

1 terraform {
2     required_version = ">= 1.5.0"
3
4     required_providers {
5         aws = {
6             source = "hashicorp/aws"
7             version = "~> 5.0"
8         }
9     }
10 }
11
12 provider "aws" {
13     region = var.aws_region
14 }
  
```

- Now Create variables and tfvars

```

1 variable "aws_region" {
2     description = "AWS Region"
3     type        = string
4 }
  
```

```

1 aws_region = "us-east-1"
  
```

- Now we can check the terraform version and proper aws configuration

```

PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform version
Terraform v1.15.5
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.15.6. You can update by downloading from https://developer.hashicorp.com/terraform/install
PS C:\Users\LENOVO\Desktop\Terraform -microservice> aws sts get-caller-identity
{
  "UserId": "928430096503",
  "Account": "928430096503",
  "Arn": "arn:aws:iam::928430096503:root"
}

PS C:\Users\LENOVO\Desktop\Terraform -microservice>
  
```

- Now create modules folder and in that ECR and create main.tf and variables.tf

```

1 resource "aws_ecr_repository" "app_repo"
2   {
3     name = "terraform-microservice"
4
5     image_scanning_configuration {
6       scan_on_push = true
7     }
8
9     force_delete = true
10
11     tags = {
12       Name = "terraform-microservice-ecr"
13     }
14   }

```

```

1 output "repository_url" {
2   value = aws_ecr_repository.app_repo.repository_url
3 }

```

- Now in root create main.tf and outputs.tf files

```

1 # ECR Module
2
3 module "ecr" {
4   source = "../modules/ecr"
5 }

```

```

1 output "ecr_repository_url" {
2   value = module.ecr.repository_url
3 }

```

- Now do terraform commands like init, validate, plan, apply and output

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform init
Initializing modules...
- ecr in modules\ecr
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

```
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform -microservice>
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform validate
Success! The configuration is valid.

PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform plan

Terraform used the selected providers to generate the following execution plan. Re
+ create

Terraform will perform the following actions:

# module.ecr.aws_ecr_repository.app_repo will be created
```

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform apply

Terraform used the selected providers to generate the following execution plan
+ create

Terraform will perform the following actions:

# module.ecr.aws_ecr_repository.app_repo will be created
+ resource "aws_ecr_repository" "app_repo" {
  + arn                = (known after apply)
  + force_delete       = true
```

```
Enter a value: yes

module.ecr.aws_ecr_repository.app_repo: Creating...
module.ecr.aws_ecr_repository.app_repo: Creation complete after 2s [id=terraform-microservice]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

ecr_repository_url = "928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice"
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform output
ecr_repository_url = "928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice"
PS C:\Users\LENOVO\Desktop\Terraform -microservice>
```

- login docker to it with password and tag the Docker Image check docker images

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> cmd /c "aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 928430096503.dkr.ecr.us-east-1.amazonaws.com"
Login Succeeded
PS C:\Users\LENOVO\Desktop\Terraform -microservice> docker tag terraform-web:v1 928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1
PS C:\Users\LENOVO\Desktop\Terraform -microservice> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1	68d70332d2dc	238MB	63.1MB	U
terraform-web:v1	68d70332d2dc	238MB	63.1MB	U
tomcat:latest	f8d2287df1bb	589MB	158MB	U

- Now Docker Image Pushed to ECR and verify image

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> docker push 928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1
The push refers to repository [928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice]
868d78dceaed: Pushed
c2e83b01f69: Pushed
72c03230f136: Pushed
a6c3a29b93ec: Pushed
8da80f8205ea: Pushed
a55ee935b5d9: Pushed
99181f19640f: Pushed
ac5e3151b8c0: Pushed
3461bb328618: Pushed
v1: digest: sha256:68d70332d2dc2d5ab80e14ec10979b24a096d7855e4c42441d99b417ce922831 size: 856
PS C:\Users\LENOVO\Desktop\Terraform -microservice> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1	68d70332d2dc	238MB	63.1MB	U
terraform-web:v1	68d70332d2dc	238MB	63.1MB	U
tomcat:latest	f8d2287df1bb	589MB	158MB	U

- Now verify in docker hub

Name	Tag	Image ID	Created	Size	Actions
tomcat	latest	f8d2287df1bb	16 days ago	589.23 MB	[Pull] [Run] [More]
928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice	v1	68d70332d2dc	2 hours ago	237.72 MB	[Pull] [Run] [More]
terraform-web	v1	68d70332d2dc	2 hours ago	237.72 MB	[Pull] [Run] [More]

- under modules folder create Networking and create main, variables output files

```
modules> networking > main.tf > resource:"aws_subnet"."public2"ig\n
1 resource "aws_vpc" "main" {
2   cidr_block      = var.vpc_cidr
3   enable_dns_hostnames = true
4   enable_dns_support = true
5
6   tags = {
7     Name = "microservice-vpc"
8   }
9 }
```

```
main.tf x variables.tf x
modules > networking > variables.tf > var
1 variable "vpc_cidr" {
2   type = string
3 }
```

```
main.tf × outputs.tf ×
modules > networking > outputs.tf >
1  output "vpc_id" {
2    | value = aws_vpc.main.id
3  }
```

- **Update Root variables.tf**

Add below your existing aws_region variable:

```
main.tf variables.tf ×
variables.tf > variable "public_subnet_2_cidr"
1  variable "aws_region" {
2    | description = "AWS Region"
3    | type       = string
4  }
5
6  variable "vpc_cidr" {
7    | type = string
8  }
9
```

- **Update Root terraform.tfvars**

```
main.tf × terraform.tfvars ×
terraform.tfvars > public_subnet_2_cidr
1  aws_region = "us-east-1"
2
3  vpc_cidr = "10.0.0.0/16"
4
```

- **Update Root main.tf**

```
main.tf ...\networking main.tf .\ ×
main.tf > module "alb"
1  # ECR Module
2
3  module "ecr" {
4    | source = "../modules/ecr"
5  }
6
7  #Networking Module
8
9  module "networking" {
10   | source = "../modules/networking"
11
12   | vpc_cidr          = var.vpc_cidr
13   | public_subnet_cidr = var.public_subnet_cidr
14   | private_subnet_cidr = var.private_subnet_cidr
15   | public_subnet_2_cidr = var.public_subnet_2_cidr
16 }
```


- Update Root outputs.tf

```

1  output "ecr_repository_url" {
2    |   value = module.ecr.repository_url
3  }
4
5  output "vpc_id" {
6    |   value = module.networking.vpc_id
7  }
8

```

- Check with Terraform init, fmt -recursive and validate all are success or not
- Update modules/networking/variables.tf

```

1  variable "aws_region" {
2    |   description = "AWS Region"
3    |   type        = string
4  }
5
6  variable "vpc_cidr" {
7    |   type = string
8  }
9
10 variable "public_subnet_cidr" {
11 |   type = string
12 }
13
14 variable "private_subnet_cidr" {
15 |   type = string
16 }

```

- Update terraform.tfvars

```

1  aws_region = "us-east-1"
2
3  vpc_cidr = "10.0.0.0/16"
4
5  public_subnet_cidr = "10.0.1.0/24"
6  private_subnet_cidr = "10.0.2.0/24"
7

```

- Update Root main.tf

```

7  #Networking Module
8
9  module "networking" {
10     source = "../modules/networking"
11
12     vpc_cidr          = var.vpc_cidr
13     public_subnet_cidr = var.public_subnet_cidr
14     private_subnet_cidr = var.private_subnet_cidr
15     public_subnet_2_cidr = var.public_subnet_2_cidr
16 }

```

- Update modules/networking/main.tf

```

main.tf ...\networking X main.tf .\
modules > networking > main.tf > resource "aws_subnet" "public2"
10
11 resource "aws_subnet" "public" {
12     vpc_id          = aws_vpc.main.id
13     cidr_block      = var.public_subnet_cidr
14     map_public_ip_on_launch = true
15
16     availability_zone = "us-east-1a"
17
18     tags = {
19         Name = "public-subnet"
20     }
21 }
22
23 resource "aws_subnet" "private" {
24     vpc_id          = aws_vpc.main.id
25     cidr_block      = var.private_subnet_cidr
26     availability_zone = "us-east-1b"
27
28     tags = {
29         Name = "private-subnet"
30     }
31 }

```

- Update modules/networking/outputs.tf

```

main.tf outputs.tf X
modules > networking > outputs.tf > output "public_subnet_2_id"
1  output "vpc_id" {
2      value = aws_vpc.main.id
3  }
4
5  output "public_subnet_id" {
6      value = aws_subnet.public.id
7  }
8
9  output "private_subnet_id" {
10     value = aws_subnet.private.id
11 }
12

```

- Now do terraform fmt -recursive and validate

```
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform fmt -recursive
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform validate
Success! The configuration is valid.

PS C:\Users\LENOVO\Desktop\Terraform -microservice> 
```

- Now Update modules/networking/main.tf

```
32
33 resource "aws_internet_gateway" "igw" {
34     vpc_id = aws_vpc.main.id
35
36     tags = {
37         Name = "microservice-igw"
38     }
39 }
```

- Update modules/networking/outputs.tf

```
main.tf  outputs.tf X
modules > networking > outputs.tf > ...
7     }
8
9     output "private_subnet_id" {
10         value = aws_subnet.private.id
11     }
12
13     output "igw_id" {
14         value = aws_internet_gateway.igw.id
15     }
```

- Verify with terraform fmt-recursive and validate if its success next adds elastic IP
- In modules/networking/main.tf, add IGW

```
main.tf X  outputs.tf
modules > networking > main.tf > resource "aws_subnet" "public2"
40
41 resource "aws_eip" "nat" {
42     domain = "vpc"
43
44     tags = {
45         Name = "nat-eip"
46     }
47
48     depends_on = [aws_internet_gateway.igw]
49 }
```

- Add NAT Gateway In modules/networking/main.tf

```

50
51 resource "aws_nat_gateway" "nat" {
52     allocation_id = aws_eip.nat.id
53     subnet_id     = aws_subnet.public.id
54
55     tags = {
56         Name = "microservice-nat"
57     }
58
59     depends_on = [aws_internet_gateway.igw]
60 }
61

```

- Update outputs Add to modules/networking/outputs.tf

```

main.tf  outputs.tf  X
modules > networking > outputs.tf > output "private_route_table_id" {
15
16
17     output "nat_gateway_id" {
18         value = aws_nat_gateway.nat.id
19     }
20

```

- Verify with terraform init, fmt – recursive and validate if its success moves to next
- Create Public route table in modules/networking/main.tf

```

main.tf  X  outputs.tf
modules > networking > main.tf > resource "aws_subnet" "public2"
62 resource "aws_route_table" "public" {
63     vpc_id = aws_vpc.main.id
64
65     route {
66         cidr_block = "0.0.0.0/0"
67         gateway_id = aws_internet_gateway.igw.id
68     }
69
70     tags = {
71         Name = "public-route-table"
72     }
73 }
74

```

- Create Private route table in modules/networking/main.tf

```
main.tf  ×  outputs.tf
modules > networking > main.tf > resource "aws_subnet" "public2"
75 resource "aws_route_table" "private" {
76     vpc_id = aws_vpc.main.id
77
78     route {
79         cidr_block      = "0.0.0.0/0"
80         nat_gateway_id = aws_nat_gateway.nat.id
81     }
82
83     tags = {
84         Name = "private-route-table"
85     }
86 }
```

- Associate Public and Private Subnets in modules/networking/main.tf

```
main.tf  ×  outputs.tf
modules > networking > main.tf > resource "aws_subnet" "public2"
87
88 resource "aws_route_table_association" "public" {
89     subnet_id      = aws_subnet.public.id
90     route_table_id = aws_route_table.public.id
91 }
92
93 resource "aws_route_table_association" "private" {
94     subnet_id      = aws_subnet.private.id
95     route_table_id = aws_route_table.private.id
96 }
97
```

- Update outputs in modules/networking/outputs

```
main.tf  outputs.tf  ×
modules > networking > outputs.tf > output "private_route_table
20
21 output "public_route_table_id" {
22     value = aws_route_table.public.id
23 }
24
25 output "private_route_table_id" {
26     value = aws_route_table.private.id
27 }
```

- Verify with terraform init, fmt-recursive, validate

- Now I created Security folder in modules and create main, variables, outputs files

The screenshot shows two Terraform files in a code editor. The top file, `variables.tf`, defines two variables: `vpc_id` and `my_ip`, both of type `string`. The bottom file, `main.tf`, defines an `aws_security_group` resource named `alb_sg`. It sets the `name` to `"alb-security-group"`, the `description` to `"ALB Security Group"`, and the `vpc_id` to `var.vpc_id`. It also defines `ingress` and `egress` rules. The `ingress` rule allows traffic on port 80 from `0.0.0.0/0`. The `egress` rule allows all traffic to port 0 from `0.0.0.0/0`.

```

modules > security > variables.tf > ...
1  variable "vpc_id" {
2    |   type = string
3  }
4
5  variable "my_ip" {
6    |   type = string
7  }
8
9
main.tf ...\networking  main.tf ...\security X
modules > security > main.tf > resource "aws_security_group" "ec
1  resource "aws_security_group" "alb_sg" {
2    name           = "alb-security-group"
3    description    = "ALB Security Group"
4    vpc_id         = var.vpc_id
5
6    ingress {
7      from_port     = 80
8      to_port       = 80
9      protocol      = "tcp"
10     cidr_blocks   = ["0.0.0.0/0"]
11   }
12
13   egress {
14     from_port      = 0
15     to_port        = 0
16     protocol       = "-1"
17     cidr_blocks    = ["0.0.0.0/0"]
18   }
19 }

```

- EC2 Security Group added below

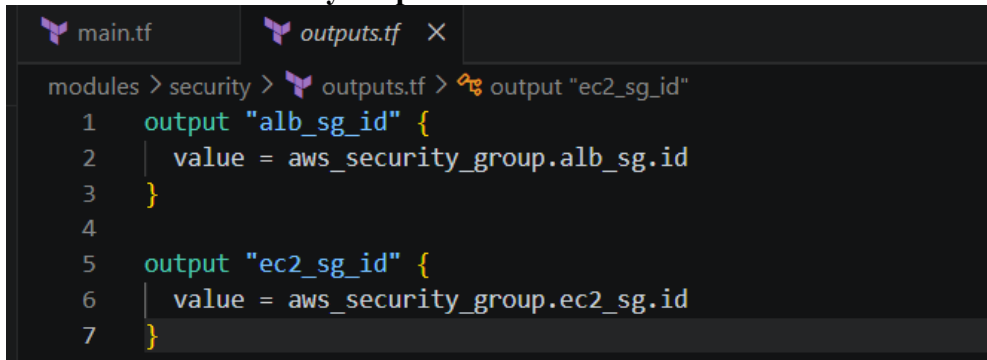
The screenshot shows the `main.tf` file in the `security` folder. It defines an `aws_security_group` resource named `ec2_sg`. It sets the `name` to `"ec2-security-group"`, the `description` to `"EC2 Security Group"`, and the `vpc_id` to `var.vpc_id`. It also defines `ingress` and `egress` rules. The first `ingress` rule allows traffic on port 80 from `0.0.0.0/0` and references the `alb_sg` resource. The second `ingress` rule allows traffic on port 22 from `var.my_ip`. The `egress` rule allows all traffic to port 0 from `0.0.0.0/0`.

```

main.tf ...\networking  main.tf ...\security X
modules > security > main.tf > resource "aws_security_group" "ec2_sg"
21 resource "aws_security_group" "ec2_sg" {
22   name           = "ec2-security-group"
23   description    = "EC2 Security Group"
24   vpc_id         = var.vpc_id
25
26   ingress {
27     from_port     = 80
28     to_port       = 80
29     protocol      = "tcp"
30     security_groups = [aws_security_group.alb_sg.id]
31   }
32
33   ingress {
34     from_port      = 22
35     to_port        = 22
36     protocol       = "tcp"
37     cidr_blocks    = [var.my_ip]
38   }
39
40   egress {
41     from_port      = 0
42     to_port        = 0
43     protocol       = "-1"
44     cidr_blocks    = ["0.0.0.0/0"]
45   }
46 }

```

- Add In modules/security/outputs.tf



```

modules > security > outputs.tf > output "ec2_sg_id"
1  output "alb_sg_id" {
2    |   value = aws_security_group.alb_sg.id
3    | }
4
5  output "ec2_sg_id" {
6    |   value = aws_security_group.ec2_sg.id
7    | }

```

- Update Root variables.tf

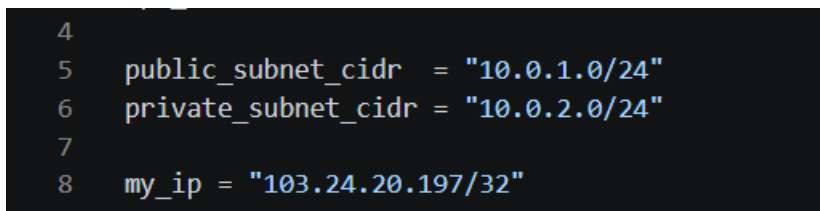


```

modules > security > variables.tf > ...
1  variable "vpc_id" {
2    |   type = string
3    | }
4
5  variable "my_ip" {
6    |   type = string
7    | }
8

```

- Update terraform.tfvars with public Ip



```

4
5  public_subnet_cidr = "10.0.1.0/24"
6  private_subnet_cidr = "10.0.2.0/24"
7
8  my_ip = "103.24.20.197/32"
9

```

- Update Root main.tf



```

**
modules > security > main.tf > module "alb"
17
18  module "security" {
19    |   source = "../modules/security"
20    |
21    |   vpc_id = module.networking.vpc_id
22    |   my_ip  = var.my_ip
23    | }
24

```

- Verify it and check everything works properly with terraform validate

- Now create EC2 folder under modules and create main, variables, output files

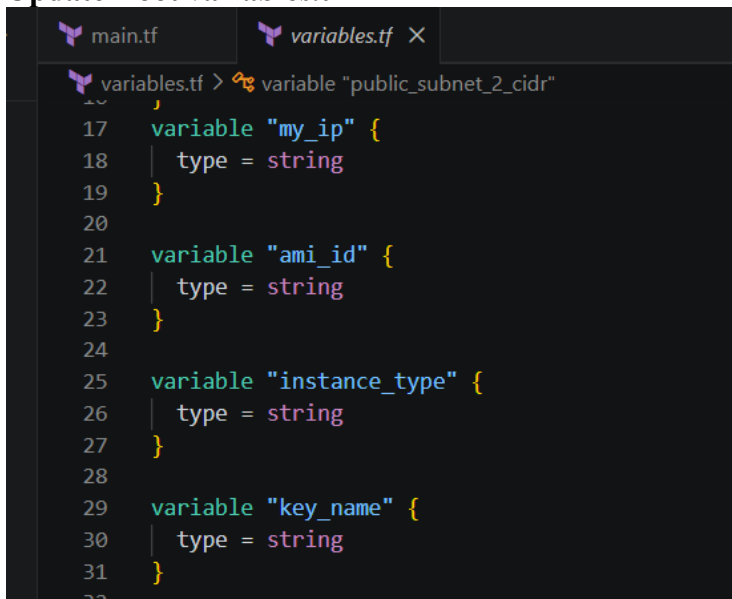


```

main.tf  variables.tf X
modules > ec2 > variables.tf > variable "user_data_file"
1  variable "ami_id" {
2    type = string
3  }
4
5  variable "instance_type" {
6    type = string
7  }
8
9  variable "private_subnet_id" {
10   type = string
11 }
12
13 variable "ec2_sg_id" {
14   type = string
15 }
16
17 variable "key_name" {
18   type = string
19 }

```

- Update Root variables.tf

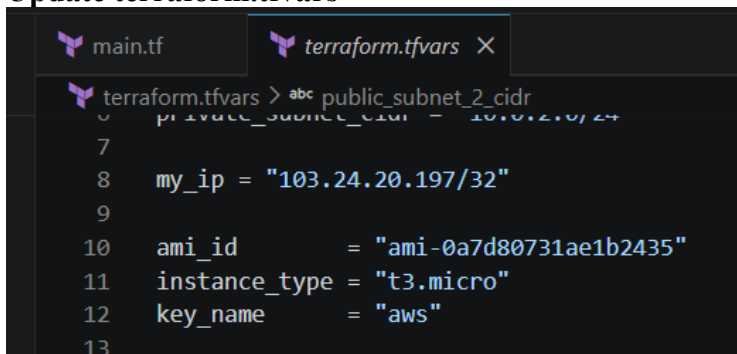


```

main.tf  variables.tf X
variables.tf > variable "public_subnet_2_cidr"
16
17 variable "my_ip" {
18   type = string
19 }
20
21 variable "ami_id" {
22   type = string
23 }
24
25 variable "instance_type" {
26   type = string
27 }
28
29 variable "key_name" {
30   type = string
31 }
32

```

- Update terraform.tfvars



```

main.tf  terraform.tfvars X
terraform.tfvars > public_subnet_2_cidr
1  public_subnet_2_cidr = "10.0.2.0/24"
2
3
4
5
6
7
8  my_ip = "103.24.20.197/32"
9
10 ami_id      = "ami-0a7d80731ae1b2435"
11 instance_type = "t3.micro"
12 key_name     = "aws"
13

```


- Now IAM Role for EC2 in modules/ec2/main.tf

```

main.tf ...\networking  main.tf ...\ec2 X
modules > ec2 > main.tf > resource "aws_instance" "app"
1  resource "aws_iam_role" "ec2_role" {
2      name = "microservice-ec2-role"
3
4      assume_role_policy = jsonencode({
5          Version = "2012-10-17"
6
7          Statement = [
8              {
9                  Action = "sts:AssumeRole"
10                 Effect = "Allow"
11
12                 Principal = {
13                     Service = "ec2.amazonaws.com"
14                 }
15             }
16         ]
17     })
18 }

```

- Attach ECR Read Policy and Create Instance Profile

```

main.tf ...\networking  main.tf ...\ec2 X
modules > ec2 > main.tf > resource "aws_instance" "app"
1  resource "aws_iam_role" "ec2_role" {
18 }
19
20 resource "aws_iam_role_policy_attachment" "ecr_readonly" {
21     role      = aws_iam_role.ec2_role.name
22     policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
23 }
24
25 resource "aws_iam_instance_profile" "ec2_profile" {
26     name = "microservice-ec2-profile"
27     role = aws_iam_role.ec2_role.name
28 }

```

- Create File in modules/ec2/outputs.tf

```

...  main.tf  outputs.tf X
modules > ec2 > outputs.tf > output "instance_id"
1  output "instance_profile_name" {
2      value = aws_iam_instance_profile.ec2_profile.name
3  }
4
5  output "instance_id" {
6      value = aws_instance.app.id
7  }

```

- Create user-data folder outside module and create install.sh file

```

main.tf  install.sh  X
user-data > $ install.sh
1  #!/bin/bash
2
3  sleep 120
4
5  apt-get update -y
6
7  apt-get install -y docker.io
8
9  systemctl enable docker
10 systemctl start docker
11
12 apt-get install -y awscli
13
14 aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 928430096503.dkr.ecr.us-east-1.amazonaws.com
15
16 docker pull 928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1
17
18 docker rm -f webapp || true
19
20 docker run -d \
21   --name webapp \
22   -p 80:80 \
23   928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:v1

```

- Now Update modules/ec2/variables.tf

```

main.tf  variables.tf ...\security  variables.tf ...\ec2  X
modules > ec2 > variables.tf > variable "user_data_file"
19  }
20
21  variable "instance_profile_name" {
22    type = string
23  }
24
25  variable "user_data_file" {
26    type = string
27  }

```

- Create EC2 Resource add in modules/ec2/main.tf

```

main.tf ...\networking  variables.tf  main.tf ...\ec2  X
modules > ec2 > main.tf > resource "aws_instance" "app"
29
30  resource "aws_instance" "app" {
31    ami            = var.ami_id
32    instance_type  = var.instance_type
33    subnet_id      = var.private_subnet_id
34    vpc_security_group_ids = [var.ec2_sg_id]
35
36    key_name = var.key_name
37
38    iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
39
40    user_data = file(var.user_data_file)
41
42    tags = {
43      Name = "microservice-server"
44    }
45  }

```

- Update modules/ec2/outputs.tf

```

main.tf  variables.tf  outputs.tf  X
modules > ec2 > outputs.tf > output "instance_id"
1  output "instance_profile_name" {
2    | value = aws_iam_instance_profile.ec2_profile.name
3  }
4
5  output "instance_id" {
6    | value = aws_instance.app.id
7  }

```

- Update root main in the EC2 module:

```

main.tf ...\networking  variables.tf  main.tf .\  X
main.tf > module "alb"
24
25  module "ec2" {
26    | source = "../modules/ec2"
27
28    | ami_id          = var.ami_id
29    | instance_type   = var.instance_type
30    | private_subnet_id = module.networking.private_subnet_id
31
32    | ec2_sg_id = module.security.ec2_sg_id
33
34    | key_name = var.key_name
35
36    | instance_profile_name = "microservice-ec2-profile"
37
38    | user_data_file = "${path.root}/user-data/install.sh"
39  }

```

- Now Create ALB in modules folder and create files
- In modules/alb/variables.tf

```

main.tf  variables.tf ...\security  variables.tf ...\alb  X
modules > alb > variables.tf > variable "public_subnet_2_id"
1  variable "vpc_id" {
2    | type = string
3  }
4
5  variable "public_subnet_id" {
6    | type = string
7  }
8
9  variable "alb_sg_id" {
10   | type = string
11 }
12
13 variable "instance_id" {
14   | type = string
15 }
16
17 variable "public_subnet_2_id" {
18   | type = string
19 }

```

- In modules/alb/outputs.tf

```

main.tf  variables.tf  outputs.tf  X
modules > alb > outputs.tf > output "alb_dns_name"
1  output "alb_dns_name" {
2      value = aws_lb.app_alb.dns_name
3  }

```

- Create Load balancer in modules/alb/main.tf

```

main.tf  ...\networking  variables.tf  main.tf ...\alb  X
modules > alb > main.tf > resource "aws_lb" "app_alb" > [ ] subnets
1  resource "aws_lb" "app_alb" {
2      name          = "microservice-alb"
3      internal      = false
4      load_balancer_type = "application"
5
6      security_groups = [var.alb_sg_id]
7
8      subnets = [
9          var.public_subnet_id,
10         var.public_subnet_2_id
11     ]
12
13     tags = {
14         Name = "microservice-alb"
15     }
16 }

```

- Create Target Group in main.tf

```

main.tf  ...\networking  variables.tf  main.tf ...\alb  X
modules > alb > main.tf > resource "aws_lb" "app_alb" > [ ] subnets
17
18 resource "aws_lb_target_group" "app_tg" {
19     name      = "microservice-tg"
20     port      = 80
21     protocol  = "HTTP"
22
23     vpc_id = var.vpc_id
24
25     health_check {
26         path = "/"
27     }
28 }

```

- Attach EC2 and Create Listener

```
main.tf ...\networking  variables.tf  main.tf ...\alb X
modules > alb > main.tf > resource "aws_lb" "app_alb" > [] subnets
29
30 resource "aws_lb_target_group_attachment" "app" {
31   target_group_arn = aws_lb_target_group.app_tg.arn
32   target_id        = var.instance_id
33   port             = 80
34 }
35
36
37 resource "aws_lb_listener" "http" {
38   load_balancer_arn = aws_lb.app_alb.arn
39   port              = 80
40   protocol          = "HTTP"
41
42   default_action {
43     type             = "forward"
44     target_group_arn = aws_lb_target_group.app_tg.arn
45   }
46 }
```

- Update in Root main.tf

```
main.tf ...\networking  variables.tf  main.tf .\ X
main.tf > module "alb"
40
41 module "alb" {
42   source = "../modules/alb"
43
44   vpc_id            = module.networking.vpc_id
45   public_subnet_id = module.networking.public_subnet_id
46   public_subnet_2_id = module.networking.public_subnet_2_id
47
48   alb_sg_id = module.security.alb_sg_id
49
50   instance_id = module.ec2.instance_id
51 }
```

- Update in Root outputs.tf

```
main.tf  variables.tf  outputs.tf X
outputs.tf > output "alb_dns_name"
7   }
8
9   output "alb_dns_name" {
10     value = module.alb.alb_dns_name
11   }
```

- Now do terraform init, validate and plan

```

PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform fmt -recursive
PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform validate
Success! The configuration is valid.

PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# module.alb.aws_lb.app_alb will be created
+ resource "aws_lb" "app_alb" {

Plan: 22 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ alb_dns_name      = (known after apply)
+ ecr_repository_url = (known after apply)
+ vpc_id            = (known after apply)

```

- Now do terraform apply

```

PS C:\Users\LENOVO\Desktop\Terraform -microservice> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# module.alb.aws_lb.app_alb will be created
+ resource "aws_lb" "app_alb" {
  + arn = (known after apply)
}

module.alb.aws_lb_listener.http: Creating...
module.alb.aws_lb_listener.http: Creation complete after 2s [id=arn:aws:elasticloadbalancing:us-east-1:928430096503:a55179f844a00ad2]

Apply complete! Resources: 22 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "microservice-alb-825124240.us-east-1.elb.amazonaws.com"
ecr_repository_url = "928430096503.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice"
vpc_id = "vpc-07c1f7c6f1a2f707b"
PS C:\Users\LENOVO\Desktop\Terraform -microservice>

```

- New instance created

Instances (1) Info

Saved filter sets

Choose filter set ▼

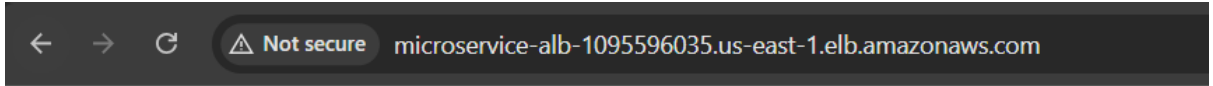
Find Instance by attribute or tag (case-sensitive)

Instance state = running X

Clear filters ▼

	Name	Instance ID	Instance state	Instance type
☐	microservice-s...	i-0cf31860b57450328	Running	t3.micro

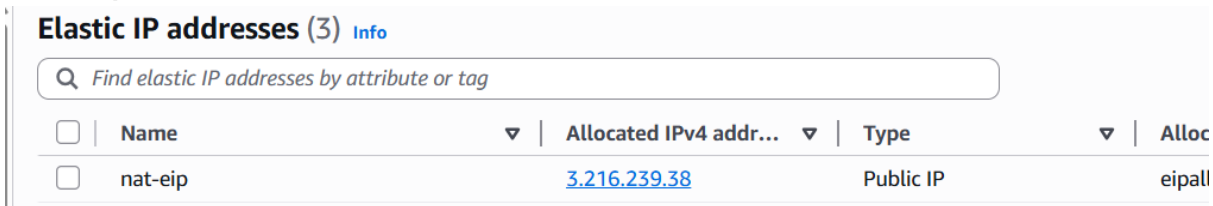
- Now check in browser with alb



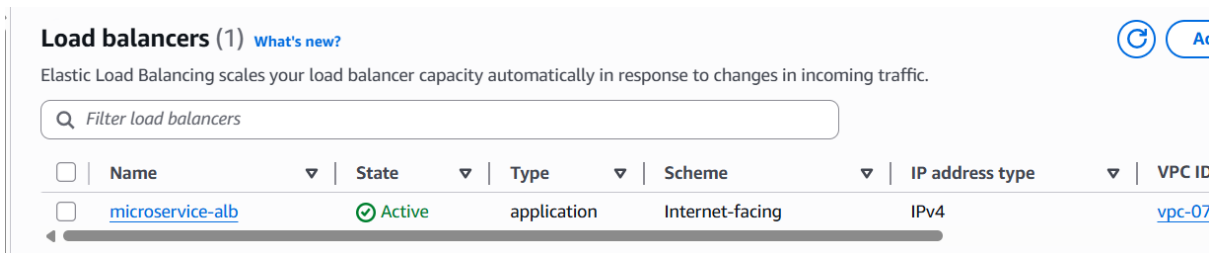
Terraform Microservice Challenge

Deployed using Docker + AWS + Terraform

- Elastic Ip created



- ALB created



- Target group created

